

1. Read an Image and Convert it to Grayscale

Aim

To read an image using Python and convert it into a grayscale image using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a new Python file.
3. Import the OpenCV (cv2) library.
4. Read the input image using cv2.imread().
5. Convert the image into grayscale using cv2.cvtColor().
6. Display both the original and grayscale images.
7. Run the program in VS Code and observe the output.

Program

```
import cv2
```

```
image = cv2.imread("image.jpg")
```

```
if image is None:
```

```
    print("Error: Image not found!")
```

```
else:
```

```
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
cv2.imshow("Original Image", image)
```

```
cv2.imshow("Grayscale Image", gray_image)
```

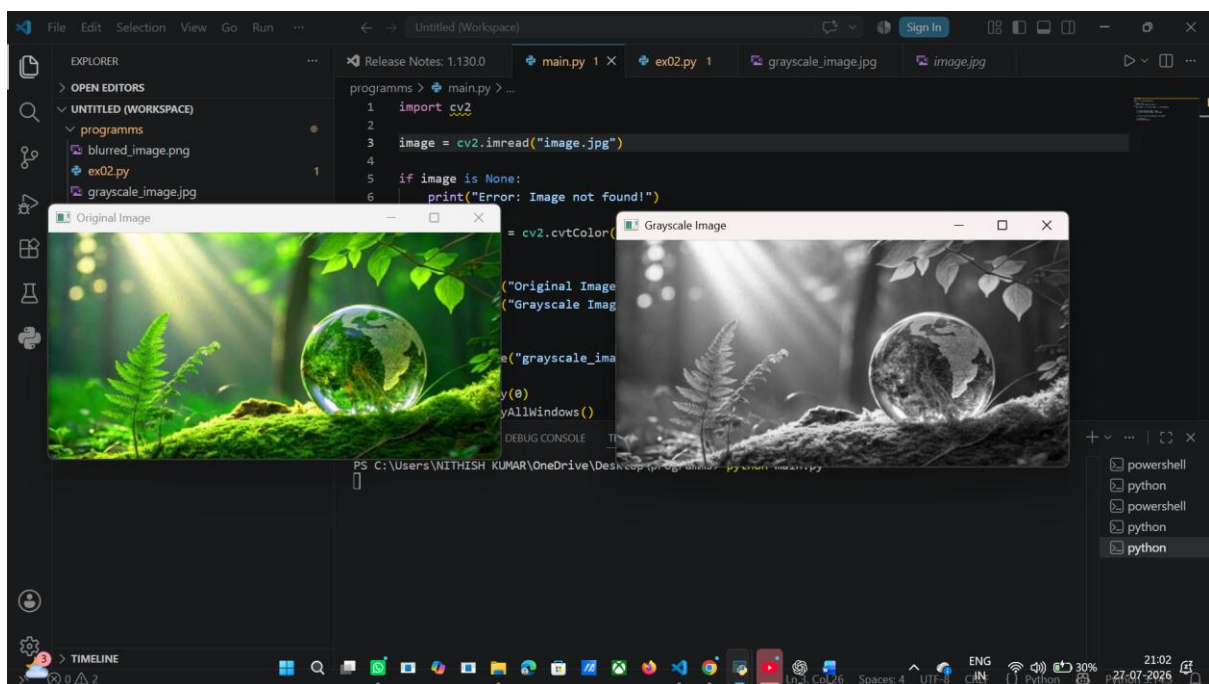
```
cv2.imwrite("grayscale_image.jpg", gray_image)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully converted from a color image into a grayscale image and displayed.



2. Read an Image and Apply Gaussian Blur

Aim

To read an image and apply Gaussian Blur using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a new Python file.
3. Import the OpenCV library.
4. Read the input image using `cv2.imread()`.
5. Apply Gaussian Blur using `cv2.GaussianBlur()`.
6. Display the original and blurred images.
7. Execute the program and verify the output.

Program

```
import cv2
```

```
# Read the image
```

```
image = cv2.imread("image.jpg")
```

```
# Check if the image is loaded
```

```
if image is None:
```

```
    print("Error: Image not found!")
```

```
else:
```

```
    # Apply Gaussian Blur
```

```
    blurred = cv2.GaussianBlur(image, (15, 15), 0)
```

```
# Display original and blurred images
```

```
cv2.imshow("Original Image", image)
```

```
cv2.imshow("Gaussian Blurred Image", blurred)
```

```
# Save the blurred image
```

```
cv2.imwrite("blurred_image.png", blurred)
```

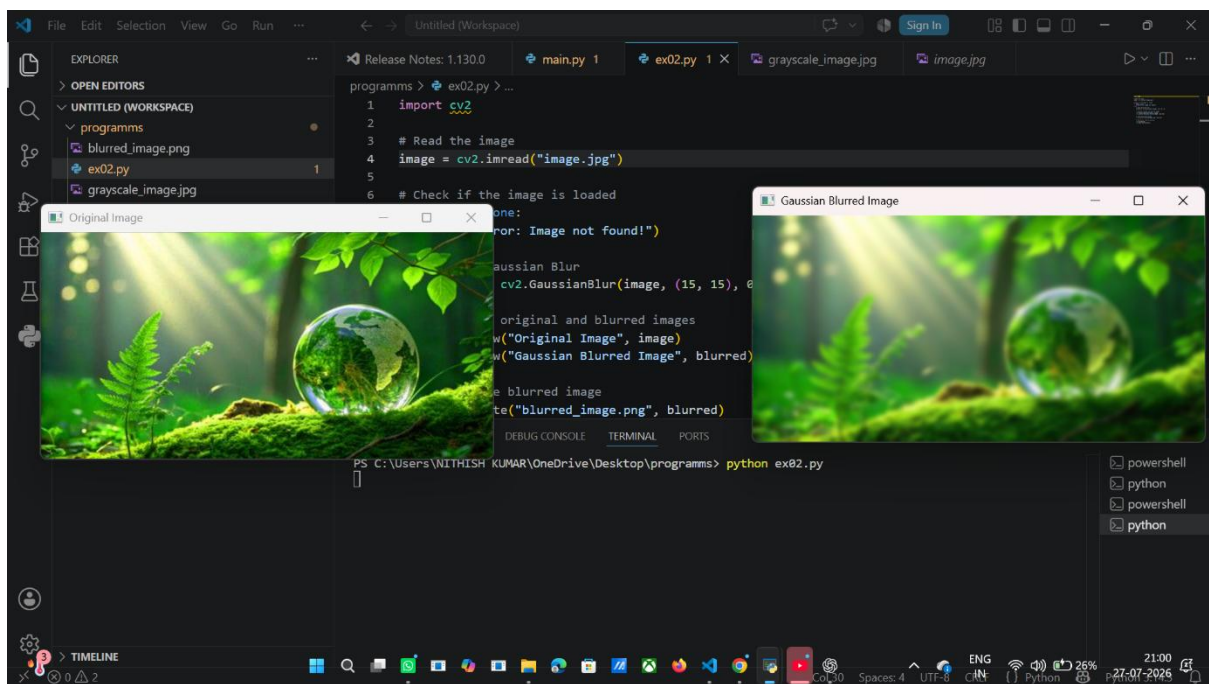
```
# Wait until a key is pressed
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully blurred using the Gaussian Blur filter.



3. Read an Image and Detect Edges using Canny

Aim

To read an image and detect its edges using the Canny edge detection algorithm.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import the OpenCV library.
4. Read the image using cv2.imread().
5. Apply the cv2.Canny() function.
6. Display the detected edges.
7. Run the program and observe the output.

Program

```
import cv2

# Read the image
image = cv2.imread("image.jpg")

# Check if the image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Convert image to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply Canny Edge Detection
    edges = cv2.Canny(gray, 100, 200)

    # Display original and outline image
    cv2.imshow("Original Image", image)
```

```
cv2.imshow("Canny Edge Detection", edges)
```

```
# Save the output image
```

```
cv2.imwrite("canny_output.png", edges)
```

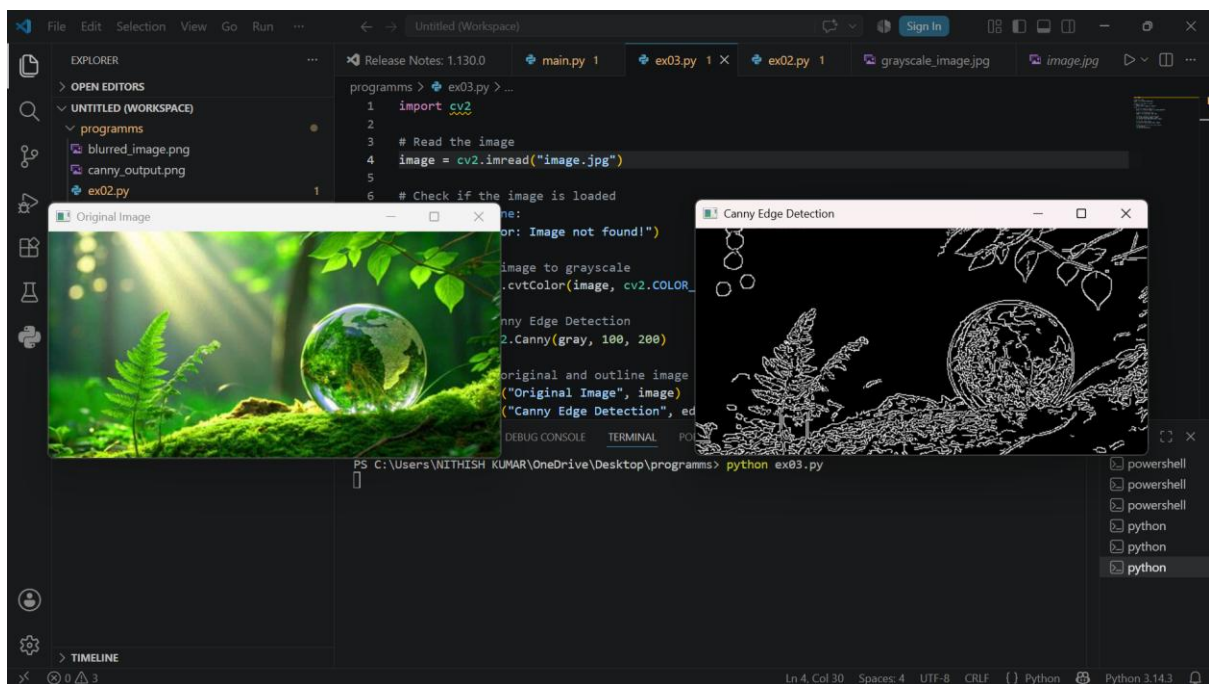
```
# Wait for a key press and close windows
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The edges of the image were successfully detected using the Canny function.



4. Read an Image and Perform Dilation

Aim

To perform image dilation using the OpenCV Dilate function.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the input image.
5. Create a kernel using NumPy.
6. Apply the cv2.dilate() function.
7. Display the dilated image.
8. Run the program.

Program

```
import cv2
import numpy as np

# Read the image
image = cv2.imread("image.jpg")

# Check if the image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Create a 5x5 kernel
    kernel = np.ones((5, 5), np.uint8)

    # Dilate the image
    dilated = cv2.dilate(image, kernel, iterations=1)
```

```
# Display original and dilated images
```

```
cv2.imshow("Original Image", image)
```

```
cv2.imshow("Dilated Image", dilated)
```

```
# Save the output image
```

```
cv2.imwrite("dilated_image.jpg", dilated)
```

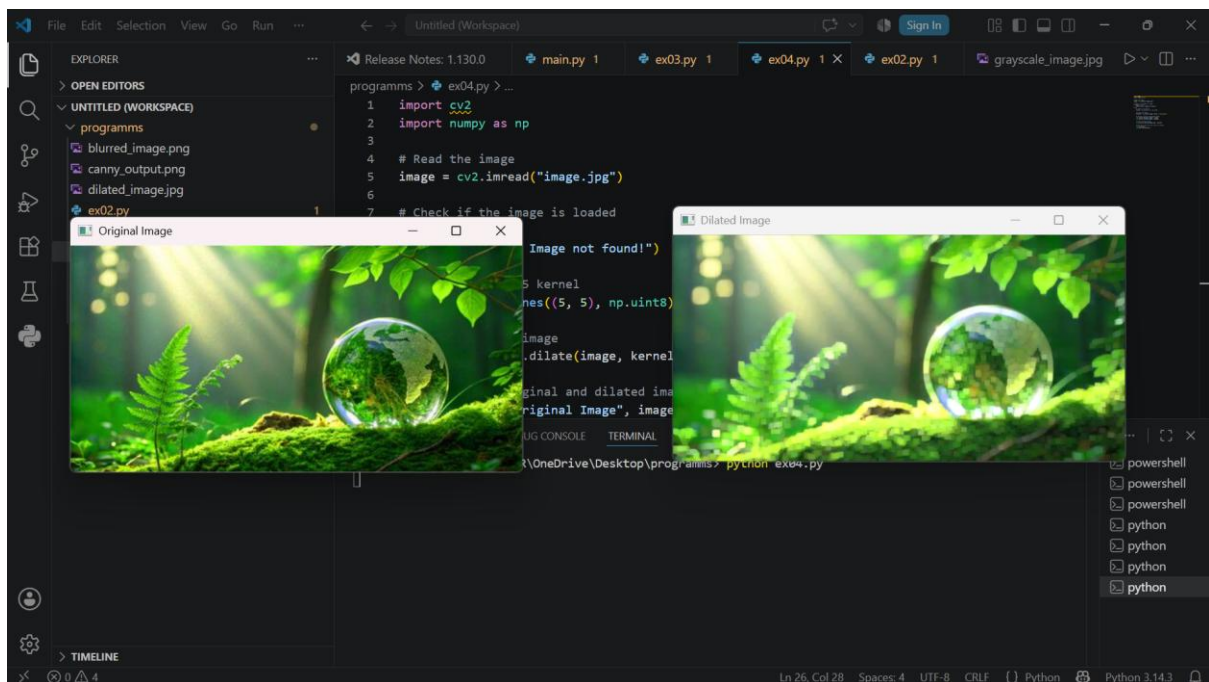
```
# Wait for a key press and close all windows
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully dilated, increasing the thickness of the objects.



5. Read an Image and Perform Erosion

Aim

To perform image erosion using the OpenCV Erode function.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Create a kernel.
6. Apply the cv2.erode() function.
7. Display the output image.
8. Execute the program.

Program

```
import cv2
import numpy as np

# Read the image
image = cv2.imread("image.jpg")

# Check if the image is loaded
if image is None:
    print("Error: Image not found!")
else:
    # Create a 5x5 kernel
    kernel = np.ones((5, 5), np.uint8)

    # Erode the image
    eroded = cv2.erode(image, kernel, iterations=1)
```

```
# Display original and eroded images
```

```
cv2.imshow("Original Image", image)
```

```
cv2.imshow("Eroded Image", eroded)
```

```
# Save the output image
```

```
cv2.imwrite("eroded_image.jpg", eroded)
```

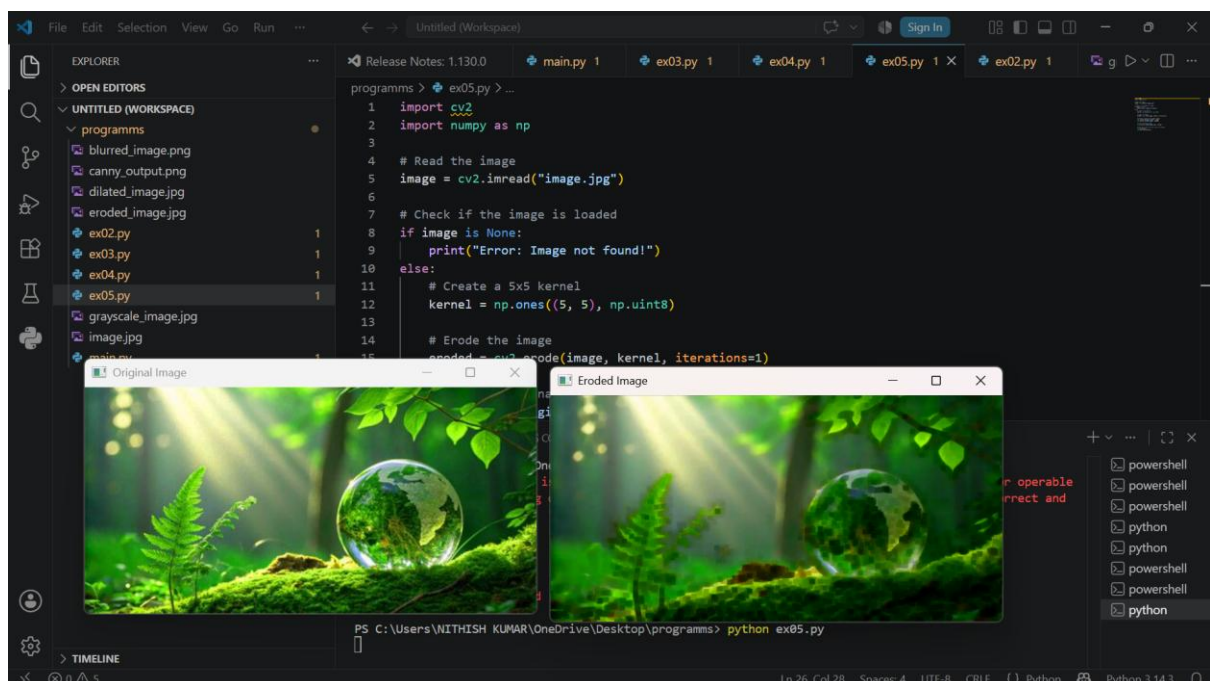
```
# Wait for a key press and close all windows
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully eroded, reducing the thickness of the objects.



6. Read a Video and Display it in Slow Motion and Fast Motion

Aim

To read a video and display it in slow motion and fast motion using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the video using `cv2.VideoCapture()`.
5. Display each frame.
6. Use `cv2.waitKey()` to control the playback speed.
7. Run the program and observe the slow and fast motion.

Program

```
import cv2
```

```
# Read the captured video
```

```
cap = cv2.VideoCapture("video.mp4")
```

```
# Check if the video is opened
```

```
if not cap.isOpened():
```

```
    print("Error: Cannot open video.")
```

```
else:
```

```
    while True:
```

```
        ret, frame = cap.read()
```

```
        if not ret:
```

```
            break
```

```
# Display the video
```

```
cv2.imshow("Video", frame)
```

```
# Press 's' for Slow Motion
```

```
key = cv2.waitKey(100) & 0xFF # Slow motion
```

```
# Press 'f' for Fast Motion
```

```
if key == ord('f'):
```

```
    cv2.waitKey(10)          # Fast motion
```

```
# Press 'q' to Quit
```

```
if key == ord('q'):
```

```
    break
```

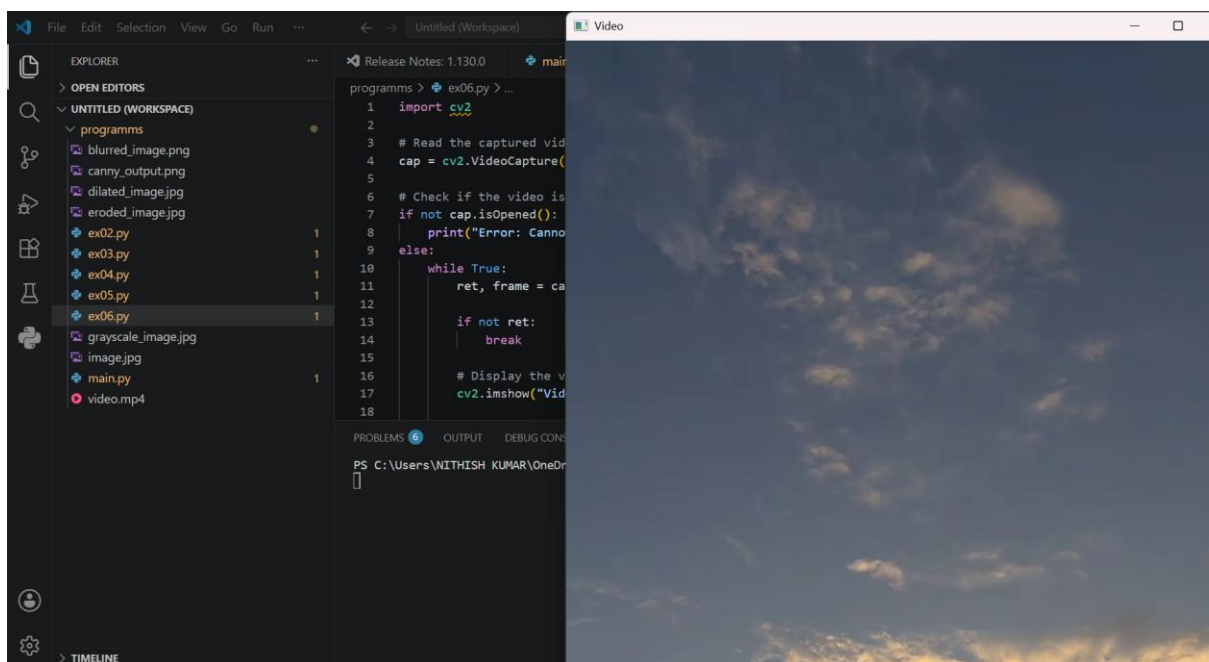
```
# Release video object and close windows
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Result

The captured video was successfully displayed in both slow motion and fast motion.



7. Capture Video from Webcam and Display it in Slow Motion and Fast Motion

Aim

To capture live video from a webcam and display it in slow motion and fast motion.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Access the webcam using `cv2.VideoCapture(0)`.
5. Capture video frames.
6. Display the frames.
7. Control the playback speed using `cv2.waitKey()`.
8. Run the program.

Program

```
import cv2
```

```
# Open the default webcam
```

```
cap = cv2.VideoCapture(0)
```

```
# Check if webcam is opened
```

```
if not cap.isOpened():
```

```
    print("Error: Cannot access webcam.")
```

```
else:
```

```
    print("Press 's' for Slow Motion")
```

```
    print("Press 'f' for Fast Motion")
```

```
    print("Press 'q' to Quit")
```

```
delay = 30 # Normal speed
```

```
while True:
```

```
ret, frame = cap.read()

if not ret:
    break

# Display webcam video
cv2.imshow("Webcam Video", frame)

key = cv2.waitKey(delay) & 0xFF

# Slow Motion
if key == ord('s'):
    delay = 100

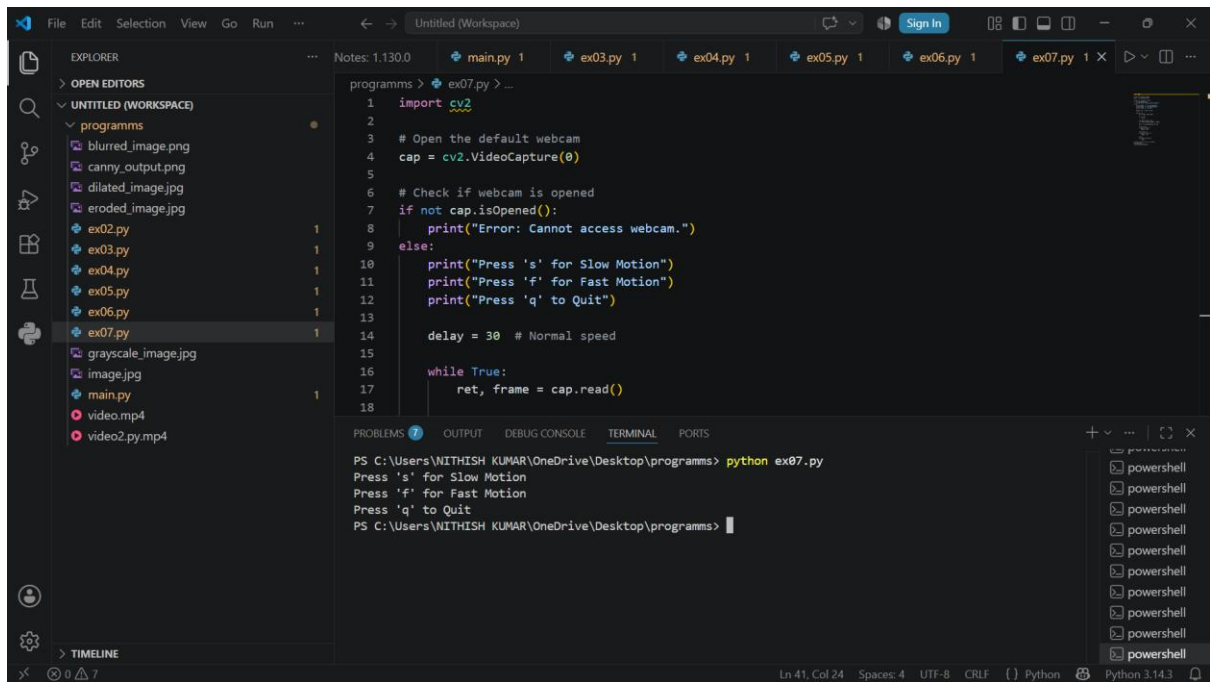
# Fast Motion
elif key == ord('f'):
    delay = 10

# Quit
elif key == ord('q'):
    break

# Release webcam and close windows
cap.release()
cv2.destroyAllWindows()
```

Result

The webcam video was successfully captured and displayed in slow motion and fast motion.



8. Scaling an Image

Aim

To resize an image into bigger and smaller sizes using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Resize the image using `cv2.resize()`.
6. Display the resized images.
7. Run the program.

Program

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
# Bigger (2x)
```

```
bigger = cv2.resize(img, None, fx=2, fy=2)
```

```
# Smaller (0.5x)
```

```
smaller = cv2.resize(img, None, fx=0.5, fy=0.5)
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Bigger", bigger)
```

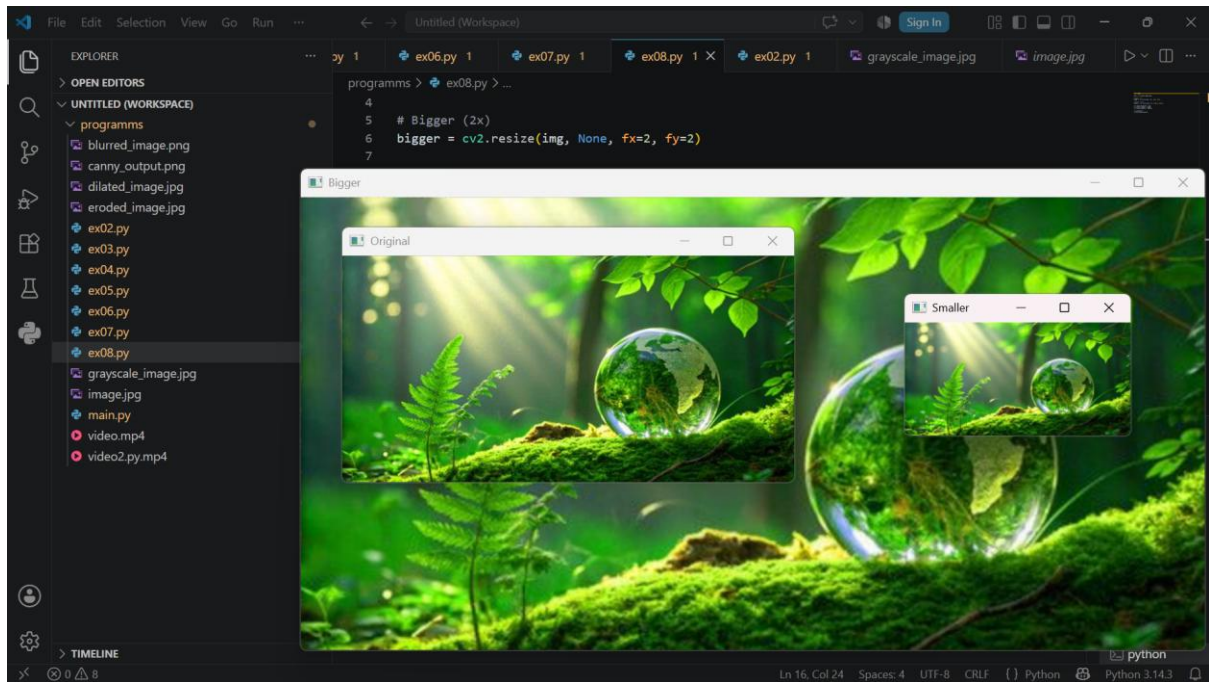
```
cv2.imshow("Smaller", smaller)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully resized into bigger and smaller dimensions.



9. Rotation of an Image

Aim

To rotate an image in clockwise and counterclockwise directions.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Generate the rotation matrix.
6. Rotate the image using `cv2.warpAffine()`.
7. Display the rotated images.
8. Execute the program.

Program

```
import cv2
```

```
img = cv2.imread("image.jpg")
```

```
# Clockwise
```

```
clockwise = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

```
# Counter Clockwise
```

```
counter = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("Clockwise", clockwise)
```

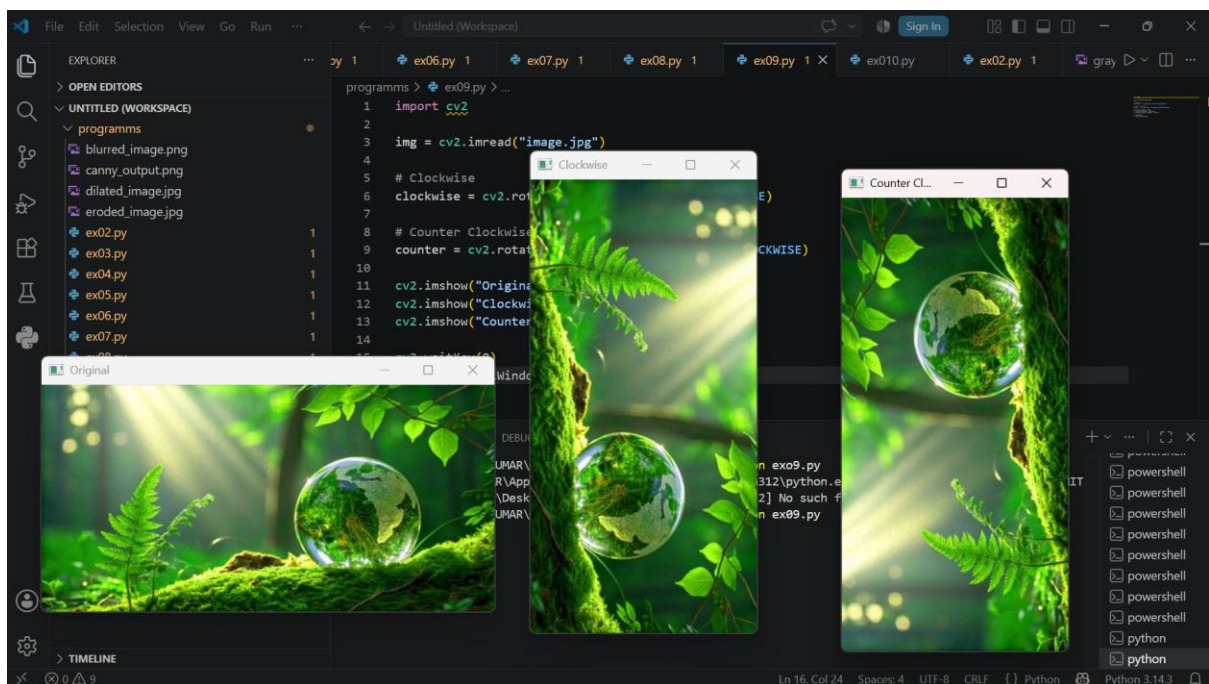
```
cv2.imshow("Counter Clockwise", counter)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Result

The image was successfully rotated in both clockwise and counterclockwise directions.



10. Translation (Moving an Image)

Aim

To move an image from one position to another using image translation.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Create a translation matrix.
6. Apply `cv2.warpAffine()`.
7. Display the translated image.
8. Run the program.

Program

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("image.jpg")
```

```
rows, cols = img.shape[:2]
```

```
# Move 100 pixels right and 50 pixels down
```

```
M = np.float32([[1, 0, 100],  
                [0, 1, 50]])
```

```
shifted = cv2.warpAffine(img, M, (cols, rows))
```

```
cv2.imshow("Original", img)
```

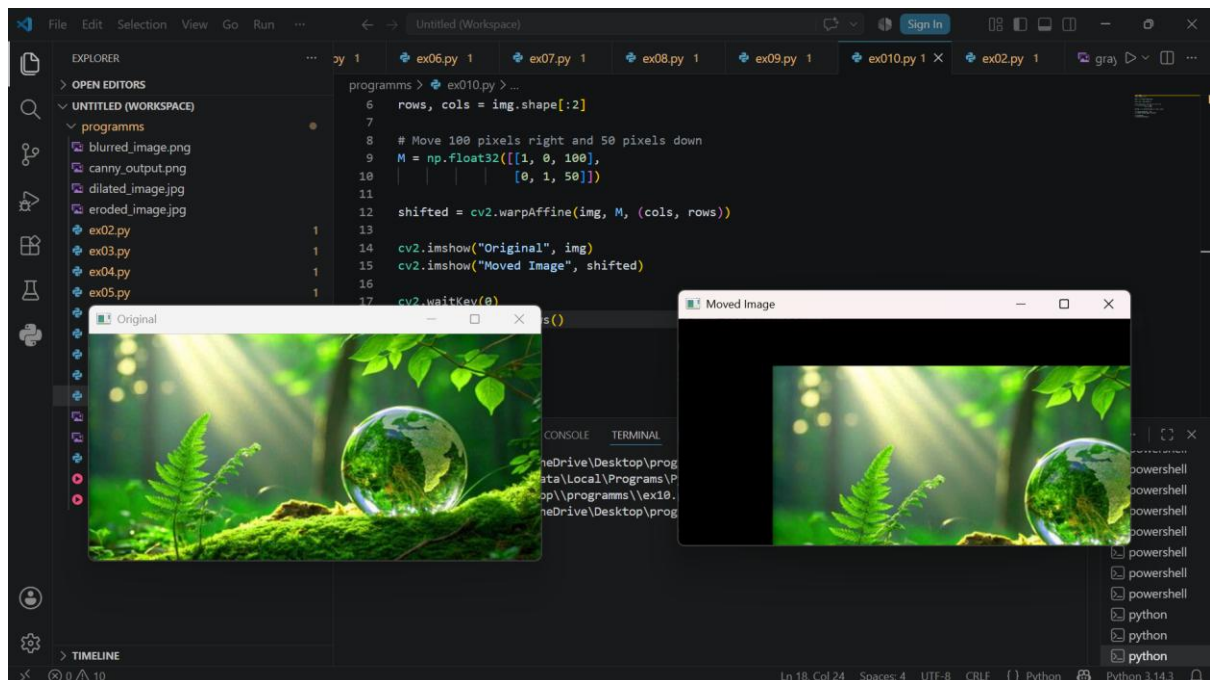
```
cv2.imshow("Moved Image", shifted)
```

`cv2.waitKey(0)`

`cv2.destroyAllWindows()`

Result

The image was successfully translated from its original position.



11. Affine Transformation

Aim

To perform an affine transformation on an image using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Select three source and destination points.
6. Compute the affine transformation matrix.
7. Apply `cv2.warpAffine()`.
8. Display the transformed image.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

rows, cols = img.shape[:2]

pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])

M = cv2.getAffineTransform(pts1, pts2)

result = cv2.warpAffine(img, M, (cols, rows))

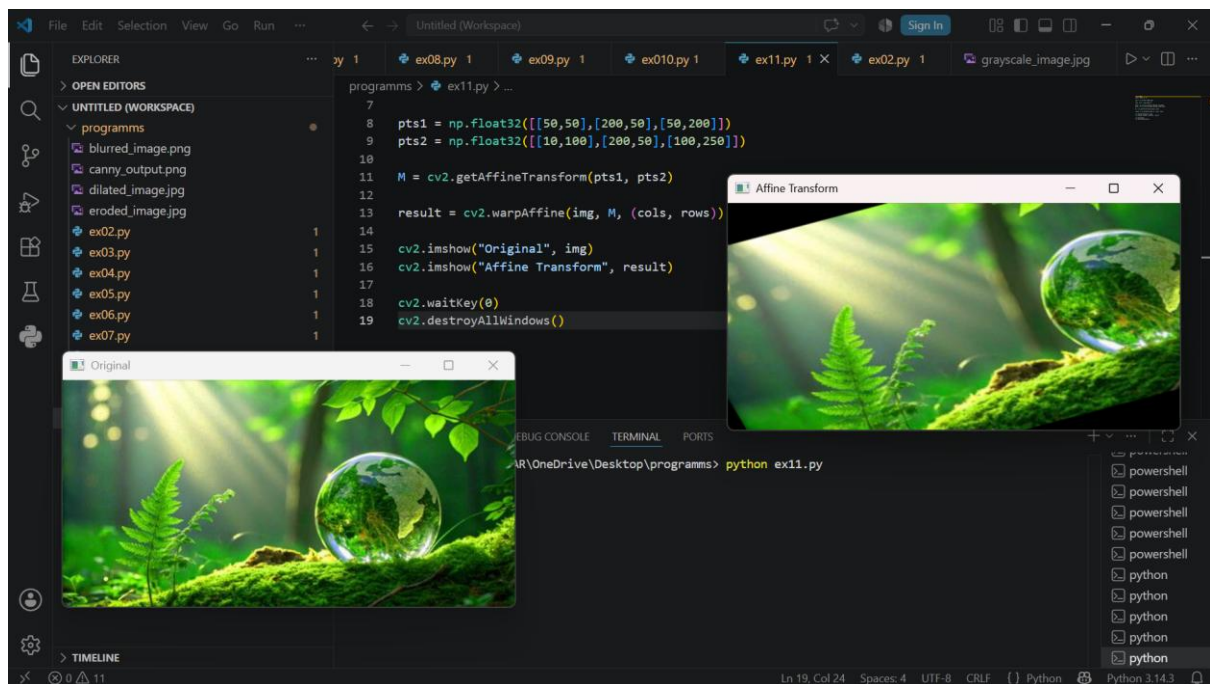
cv2.imshow("Original", img)
cv2.imshow("Affine Transform", result)
```

`cv2.waitKey(0)`

`cv2.destroyAllWindows()`

Result

The affine transformation was successfully performed on the image.



12. Perspective Transformation on an Image

Aim

To perform perspective transformation on an image.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the image.
5. Select four source and destination points.
6. Generate the perspective matrix.
7. Apply `cv2.warpPerspective()`.
8. Display the transformed image.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

rows, cols = img.shape[:2]

pts1 = np.float32([[50,50],[300,50],[50,300],[300,300]])
pts2 = np.float32([[0,0],[300,0],[100,300],[250,300]])

M = cv2.getPerspectiveTransform(pts1, pts2)

result = cv2.warpPerspective(img, M, (cols, rows))

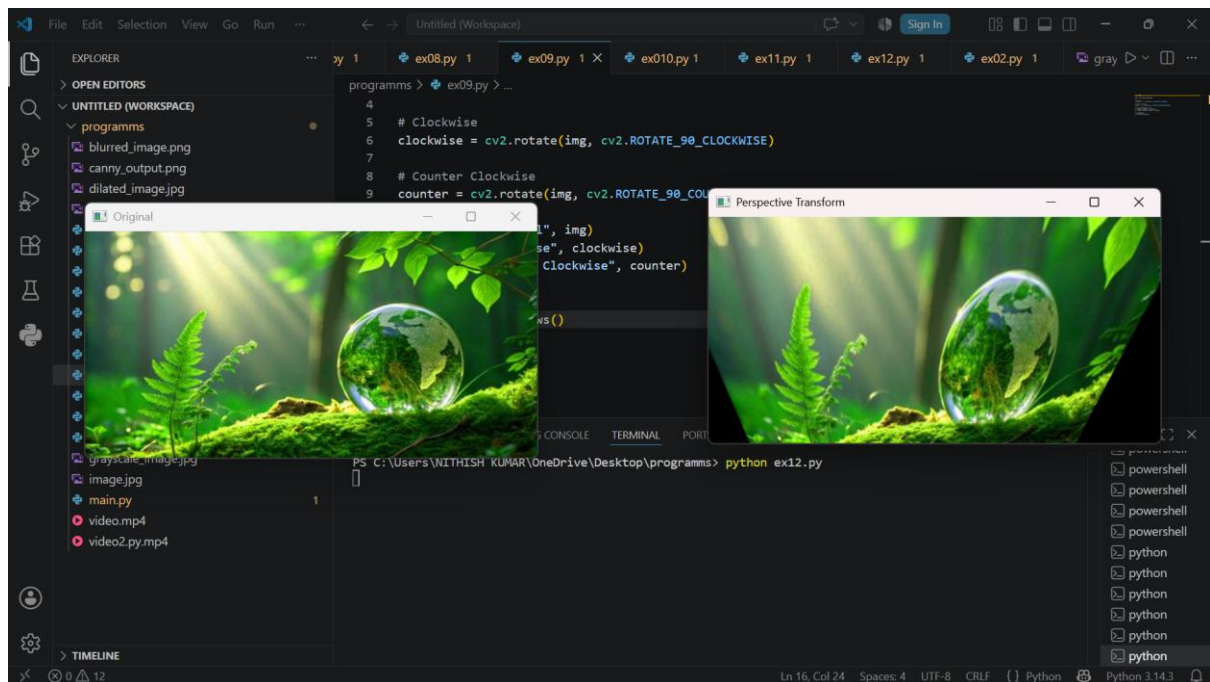
cv2.imshow("Original", img)
cv2.imshow("Perspective Transform", result)
```


`cv2.waitKey(0)`

`cv2.destroyAllWindows()`

Result

The perspective transformation was successfully applied to the image.



13. Perspective Transformation on a Video

Aim

To perform perspective transformation on a video using OpenCV.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Open the video using `cv2.VideoCapture()`.
5. Read each frame.
6. Apply `cv2.warpPerspective()` to each frame.
7. Display the transformed frames.
8. Run the program.

Program

```
import cv2
```

```
import numpy as np
```

```
cap = cv2.VideoCapture("video2.py.mp4")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    h, w = frame.shape[:2]
```

```
    pts1 = np.float32([[50,50],[w-50,50],[50,h-50],[w-50,h-50]])
```

```
pts2 = np.float32([[0,0],[w,0],[100,h],[w-100,h]])
```

```
M = cv2.getPerspectiveTransform(pts1, pts2)
```

```
transformed = cv2.warpPerspective(frame, M, (w,h))
```

```
cv2.imshow("Original Video", frame)
```

```
cv2.imshow("Perspective Video", transformed)
```

```
if cv2.waitKey(25) & 0xFF == ord('q'):
```

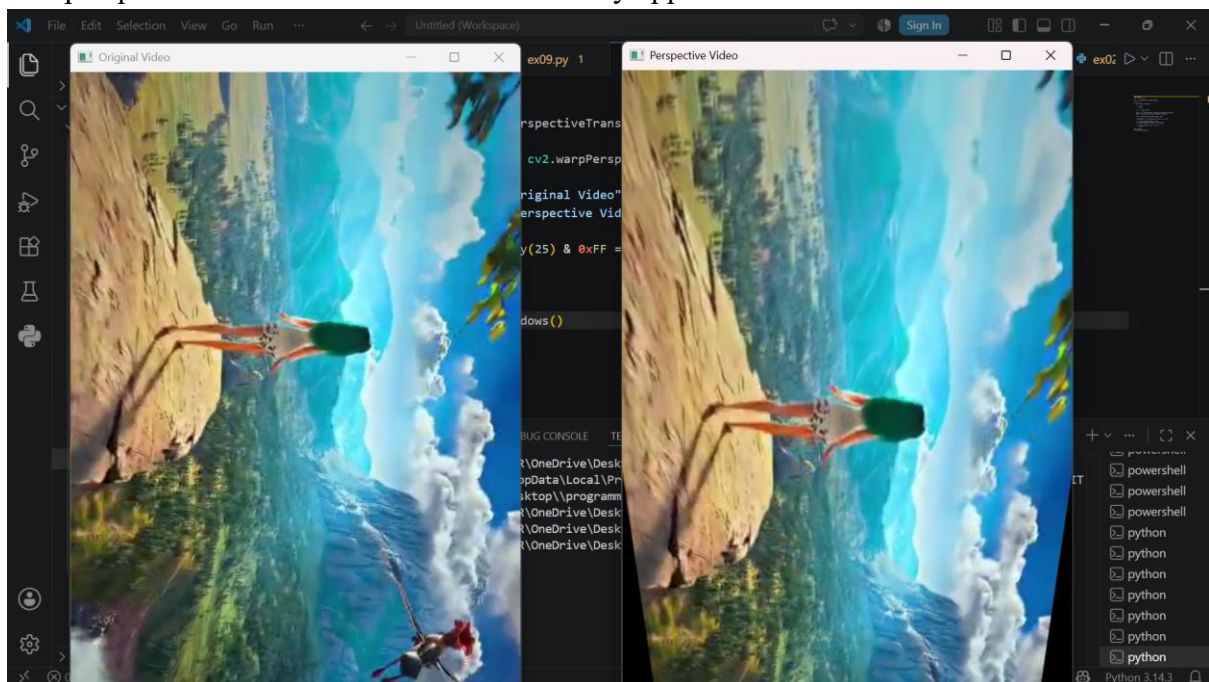
```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

Result

The perspective transformation was successfully applied to the video frames.



14. Transformation using Homography Matrix

Aim

To perform image transformation using a Homography matrix.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import OpenCV.
4. Read the input image.
5. Select matching source and destination points.
6. Compute the Homography matrix using `cv2.findHomography()`.
7. Apply the transformation using `cv2.warpPerspective()`.
8. Display the output image.

Program

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("image.jpg")
```

```
rows, cols = img.shape[:2]
```

```
src = np.float32([[50,50],[300,50],[50,300],[300,300]])
```

```
dst = np.float32([[10,100],[250,50],[100,250],[300,300]])
```

```
H, status = cv2.findHomography(src, dst)
```

```
result = cv2.warpPerspective(img, H, (cols, rows))
```

```
cv2.imshow("Original", img)
```

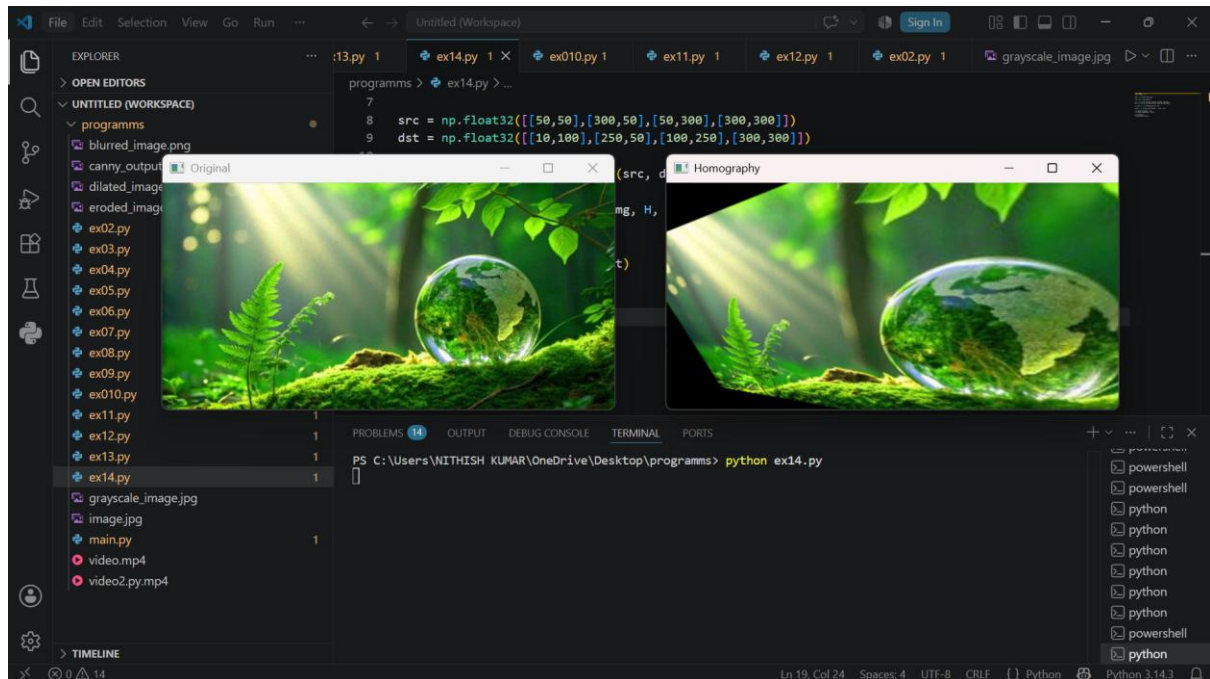
```
cv2.imshow("Homography", result)
```

`cv2.waitKey(0)`

`cv2.destroyAllWindows()`

Result

The image was successfully transformed using the Homography matrix.



15. Transformation using Direct Linear Transformation (DLT)

Aim

To perform image transformation using the Direct Linear Transformation (DLT) method.

Procedure

1. Open Visual Studio Code.
2. Create a Python file.
3. Import the required libraries.
4. Read the input image.
5. Define corresponding source and destination points.
6. Compute the transformation matrix using the DLT method.
7. Apply the transformation to the image.
8. Display the transformed image and run the program.
- 9.

Program

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

rows, cols = img.shape[:2]

src = np.float32([[50,50],[300,50],[50,300],[300,300]])
dst = np.float32([[0,0],[250,20],[50,300],[300,300]])

# Homography using DLT
H, status = cv2.findHomography(src, dst, 0)

output = cv2.warpPerspective(img, H, (cols, rows))
```

```
cv2.imshow("Original", img)
```

```
cv2.imshow("DLT Transformation", output)
```

```
cv2.waitKey(0)
```

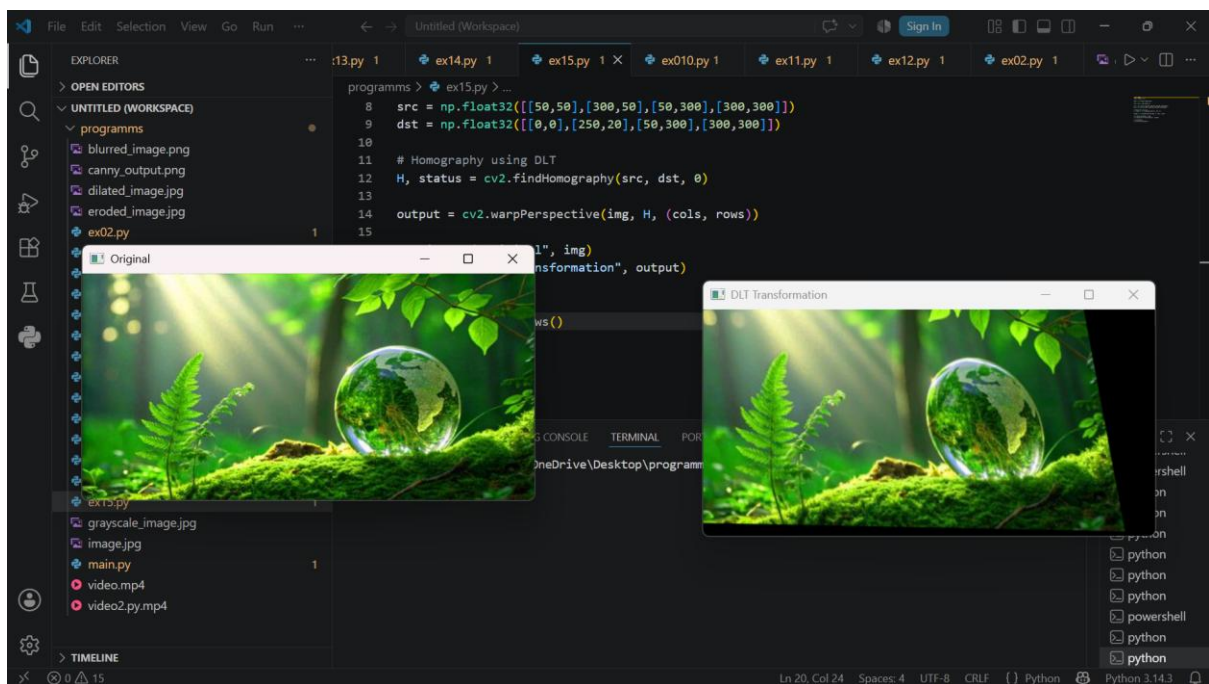
```
cv2.destroyAllWindows()
```

Result

The image was successfully transformed using the Direct Linear Transformation (DLT) method.

Result

The image was successfully transformed using the Homography matrix.



16. Edge Detection using Canny Method

Aim

To detect the edges in an image using the Canny edge detection algorithm.

Program

```
import cv2

# Read the image in grayscale
img = cv2.imread("image.jpg", 0)

# Apply Canny edge detection
edges = cv2.Canny(img, 100, 200)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Canny Edge Detection", edges)

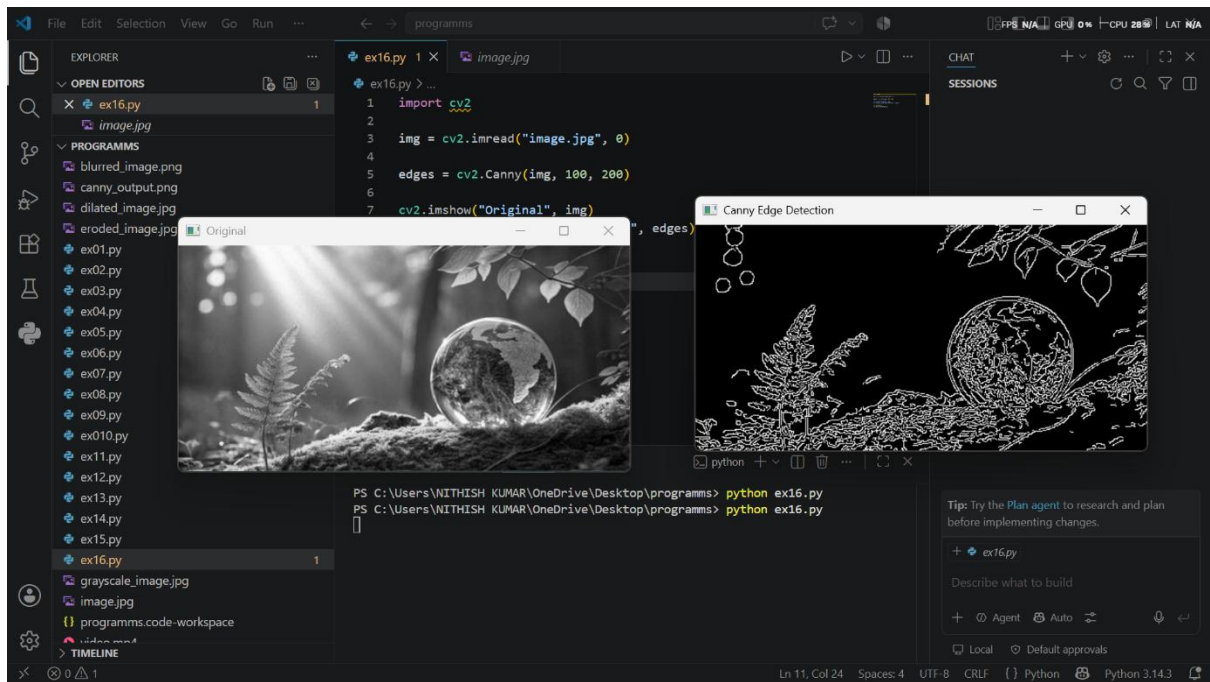
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Import the OpenCV library.
2. Read the input image in grayscale.
3. Apply the `cv2.Canny()` function with suitable threshold values.
4. Display the original and edge-detected images.
5. Close all windows after a key press.

Result

The edges of the input image were successfully detected using the Canny edge detection method.



17. Edge Detection using Sobel Matrix (X-axis)

Aim

To detect vertical edges using the Sobel X operator.

Program

```
import cv2

img = cv2.imread("image.jpg", 0)

sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)

cv2.imshow("Original", img)
cv2.imshow("Sobel X", sobelx)

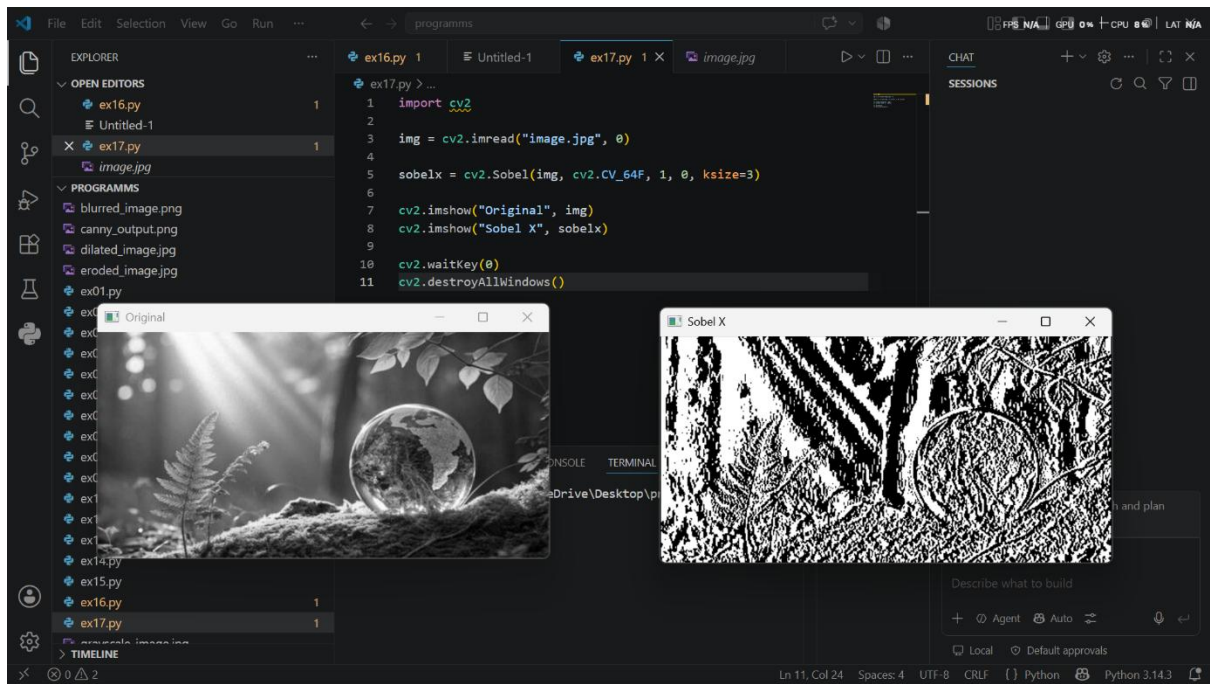
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Convert it to grayscale.
3. Apply Sobel operator in X-direction.
4. Display the output.

Result

Vertical edges were successfully detected using Sobel X.



18. Edge Detection using Sobel Matrix (Y-axis)

Aim

To detect horizontal edges using the Sobel Y operator.

Program

```
import cv2

img = cv2.imread("image.jpg", 0)

sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

cv2.imshow("Original", img)
cv2.imshow("Sobel Y", sobely)

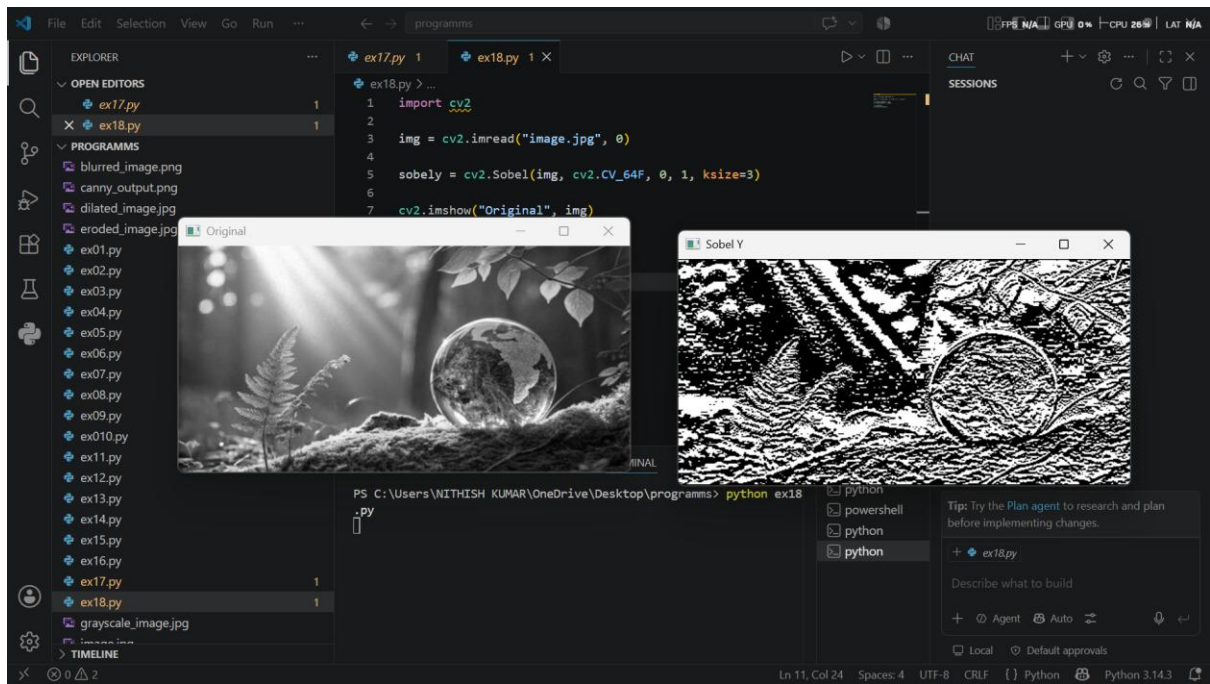
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Convert it to grayscale.
3. Apply Sobel operator in Y-direction.
4. Display the output.

Result

Horizontal edges were successfully detected using Sobel Y.



19. Edge Detection using Sobel Matrix (X and Y-axis)

Aim

To detect edges in both X and Y directions using Sobel operators.

Program

```
import cv2

img = cv2.imread("image.jpg", 0)

sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

sobelxy = cv2.addWeighted(sobelx, 0.5, sobely, 0.5, 0)

cv2.imshow("Original", img)
cv2.imshow("Sobel XY", sobelxy)

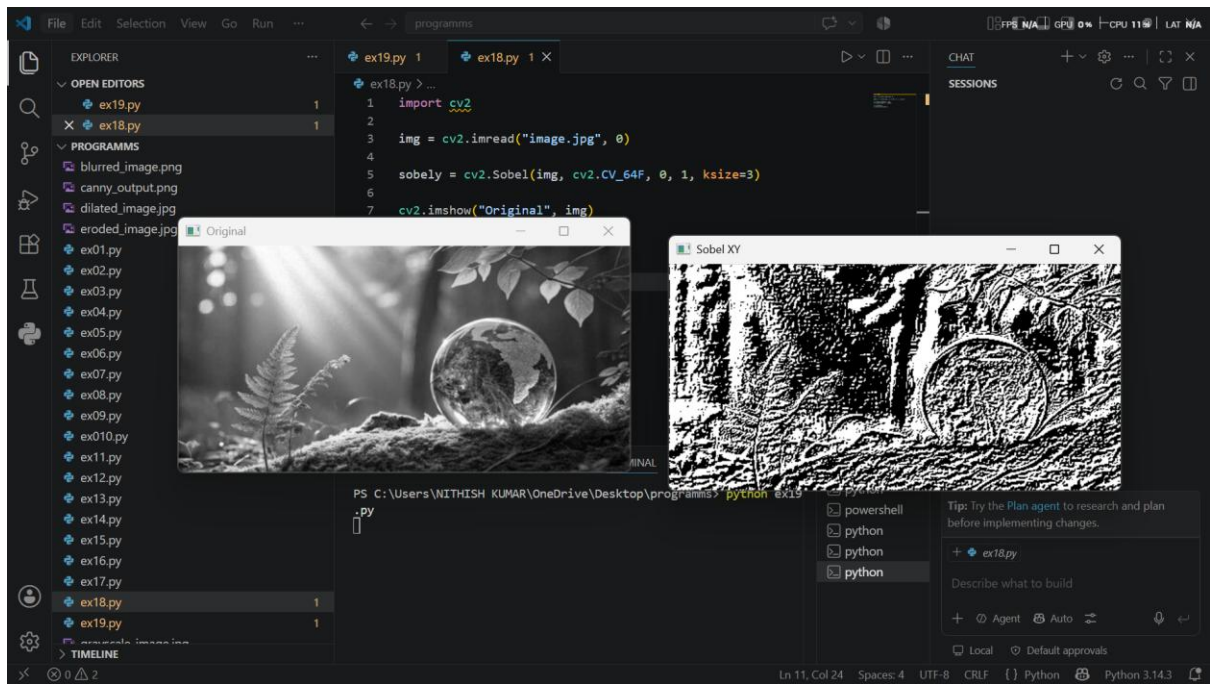
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Procedure

1. Read the image.
2. Convert it to grayscale.
3. Apply Sobel X and Sobel Y.
4. Combine both outputs.
5. Display the result.

Result

Edges in both horizontal and vertical directions were successfully detected.



20. Sharpening using Laplacian Mask (Negative Center Coefficient)

Aim

To sharpen the image using a Laplacian mask with a negative center coefficient.

Mask Used

0 1 0

1 -4 1

0 1 0

Program

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread("image.jpg")
```

```
kernel = np.array([[0, 1, 0],
```

```
                  [1,-4, 1],
```

```
                  [0, 1, 0]])
```

```
laplacian = cv2.filter2D(img, -1, kernel)
```

```
sharpened = cv2.subtract(img, laplacian)
```

```
cv2.imshow("Original Image", img)
```

```
cv2.imshow("Laplacian", laplacian)
```

```
cv2.imshow("Sharpened Image", sharpened)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Procedure

1. Read the input image.

2. Define the Laplacian mask.
3. Apply the mask using convolution.
4. Subtract the Laplacian image from the original image.
5. Display the sharpened image.

Result

The image was successfully sharpened, and edge details were enhanced using the Laplacian mask

